Dictionaries in Python 3

Module: Core Python · Collections
Previous module: Tuples and Sets
Next module: Data Structures Overview

Module Outcome:

- Look up a contact, SKU, or roll number by **key** — not by scanning a list
- Add, update, and guard missing keys so a Apollo Hospitals form does not explode
- Nest a dict when one person has many fields (name, marks, phone)

A **dictionary** is like a real dictionary or phone contacts app:

- You look up a **word (key)** and get a **meaning (value)**.
- Keys must be **unique** — one phone number cannot map to two different people in the same dict.
- Values can be anything: string, number, list, or even another dictionary.

👤 Employee 📦 Order 🏦 Bank 💰 Payroll 🗂️ CRM

Where used?
- **Employee Management Portal** — `{emp_id: profile}` lookups
- **Swiggy** — product by SKU, order JSON with nested items
- **HDFC Bank** — account record fields
- **PhonePe** — UPI txn metadata
- College apps — student dict (`roll` , `marks` , `grade`)
- **Apollo Hospitals** — contact book, ticket fields from APIs

Examples from daily work:

Key	Value	Scenario
`"emp_id"`	`101`	Employee record
`"email"`	`"aisha@company.com"`	Login profile
`"price"`	`1499.00`	E-commerce product
`"items"`	`[{"sku": "P1", "qty": 2}]`	Order JSON from API

When you fetch data from a REST API, you almost always handle **dicts** (and lists of dicts).

Concepts and Definitions

Term	Definition
Dictionary (`dict`)	Mutable mapping of unique keys to values.
Key	Lookup identifier — str, int, tuple (hashable), etc.

Term	Definition
Value	Data stored for a key — any type.
Key-value pair	One entry: `"name": "Ananya"`.
Hashable	Can be used as dict key (not list/dict).
Nested dict	Dict inside dict — models JSON documents.
`.get(key, default)`	Safe read; no KeyError if missing.
JSON	JavaScript Object Notation — text format; Python dict ↔ JSON in APIs.

Step-by-Step Notes with Annotated Python 3 Code

Step 1 — Create and read (CRUD: Create, Read)

```
# CREATE — literal or dict()
employee = {
    "emp_id": 101,
    "name": "Ananya Sharma",
    "department": "Engineering",
    "salary": 85000,
}

empty_cart = {}
product = dict(sku="P001", name="Wireless Mouse", price=799)

# READ — bracket notation
print(employee["name"])             # Ananya Sharma
print(product["price"])             # 799

# READ — safe with .get()
print(employee.get("email"))          # None — key missing, no crash
print(employee.get("email", "N/A"))   # N/A — default provided

# KeyError if missing with brackets:
# print(employee["email"])          # KeyError
```

Note:
- Wrong: Using a list as a key → `TypeError: unhashable type: 'list'`; `d["missing"]` without a check → `KeyError`; duplicate keys in a literal (last value wins silently).
- Prefer: String or tuple keys; `.get()` or `if key in d`; avoid duplicate keys (`{"a": 1, "a": 2}` → `a` is 2).

Create a student dict, print name by key, and use `.get` for a missing phone with a default.

```
student = {"name": "Priya", "roll": "2026001", "marks": 88}
print(student["name"])
print(student.get("phone", "N/A"))
```

Expected Output:

```
Priya
N/A
```

Remember: Prefer `.get("key", default)` when a field might be missing — brackets `d["key"]` raise `KeyError`.

Create a Swiggy product with `dict()` and print its price.

```
product = dict(sku="P10", name="Chai Mug", price=299)
print(product["price"])
```

Expected Output:

```
299
```

Step 2 — Update and add (CRUD: Update)

```
contact = {"name": "Dev Kumar", "phone": "9876543210"}

# UPDATE existing key
contact["phone"] = "9876500000"

# ADD new key
contact["email"] = "dev@example.com"
contact["city"] = "Bengaluru"

print(contact)
# {'name': 'Dev Kumar', 'phone': '9876500000', 'email': 'dev@example.com', 'city': 'Bengaluru'}

# update() — merge another dict
extra = {"company": "KSR Tech", "role": "Developer"}
contact.update(extra)
```

Update phone, add email and city to a contact dict, and print the full dict.

```
contact = {"name": "Ravi", "phone": "9876543210"}
contact["phone"] = "9876500000"
contact["email"] = "ravi@example.com"
contact["city"] = "Bengaluru"
print(contact)
```

Merge role and company into a contact dict with `update()` and print the result.

```
contact = {"name": "Ravi", "phone": "9876543210"}
contact.update({"role": "Intern", "company": "KSR Tech"})
print(contact)
```

Step 3 — Delete (CRUD: Delete)

```
expense = {"date": "2025-07-01", "category": "Food", "amount": 450}

# DELETE one key
del expense["category"]

# pop — remove and return value
amount = expense.pop("amount", 0)     # 450; default 0 if missing
print(amount, expense)                # 450 {'date': '2025-07-01'}

# clear all
expense.clear()                       # {}
```

Pop the amount from an expense dict and print the returned value and remaining dict.

```
expense = {"date": "01-08-2026", "category": "Food", "amount": 450}
amount = expense.pop("amount")
print(amount)
print(expense)
```

Expected Output:

```
450
{'date': '01-08-2026', 'category': 'Food'}
```

Delete category from an expense, clear all keys, and print the empty dict.

```
expense = {"date": "01-08-2026", "category": "Food", "amount": 450}
del expense["category"]
expense.clear()
print(expense)
```

Expected Output:

```
{}
```

Step 4 — Keys, values, items

```
product = {"sku": "P002", "name": "Keyboard", "price": 1499, "stock": 25}

print(list(product.keys()))           # ['sku', 'name', 'price', 'stock']
print(list(product.values()))         # ['P002', 'Keyboard', 1499, 25]

# items() — pairs for looping
for key, value in product.items():
    print(f"{key}: {value}")
```

Loop subject-score pairs in a marks dict and print each line.

```
marks = {"Python": 88, "SQL": 76}
for subject, score in marks.items():
```

```
    print(f"{subject}: {score}")
```

Expected Output:

```
Python: 88
SQL: 76
```

Print lists of keys and values from a product dict.

```
product = {"sku": "P10", "name": "Chai Mug", "price": 299}
print(list(product.keys()))
print(list(product.values()))
```

Step 5 — Iterate and build new dicts

```
# Word frequency from review text (e-commerce)
reviews = ["good", "fast", "good", "quality", "fast", "good"]
frequency = {}

for word in reviews:
    frequency[word] = frequency.get(word, 0) + 1

print(frequency)                      # {'good': 3, 'fast': 2, 'quality': 1}

# Dict comprehension — 10% discount prices
prices = {"mouse": 799, "keyboard": 1499, "hub": 599}
sale_prices = {k: round(v * 0.9, 2) for k, v in prices.items()}
print(sale_prices)
```

Note:

- Wrong: Modifying a dict while iterating — RuntimeError or skipped keys.
- Prefer: Iterate over a copy: `list(d.items())`.

Count word frequency in a list using `.get` and print the frequency dict.

```
words = ["good", "fast", "good", "ok", "fast"]
frequency = {}
for word in words:
    frequency[word] = frequency.get(word, 0) + 1
print(frequency)
```

Expected Output:

```
{'good': 2, 'fast': 2, 'ok': 1}
```

Build discounted prices at 10% off with a dict comprehension and print the result.

```
prices = {"mouse": 799, "hub": 599}
discounted = {k: round(v * 0.9, 2) for k, v in prices.items()}
print(discounted)
```

Expected Output:

```
{'mouse': 719.1, 'hub': 539.1}
```

Step 6 — Nested dictionaries

```
# Employee with nested address
employee = {
    "emp_id": 101,
    "name": "Meera Iyer",
    "address": {
        "city": "Chennai",
        "pincode": "600001",
        "country": "India",
    },
    "skills": ["Python", "SQL", "Git"],
}

# READ nested
print(employee["address"]["city"])    # Chennai
print(employee["skills"][0])          # Python

# UPDATE nested
employee["address"]["pincode"] = "600002"
```

Note:
- Wrong: Shared nested dict reference — changing a shallow copy affects the original.
- Prefer: `copy.deepcopy()` for nested structures.

Read nested city and first skill from an employee dict and print them.

```
employee = {
    "name": "Meera",
    "address": {"city": "Pune", "pincode": "411001"},
    "skills": ["Python", "SQL"],
}
print(employee["address"]["city"])
print(employee["skills"][0])
```

Update a nested address pincode and print the address sub-dict.

```
employee = {
    "name": "Meera",
    "address": {"city": "Pune", "pincode": "411001"},
}
employee["address"]["pincode"] = "411014"
print(employee["address"])
```

Step 7 — JSON-like order structure (API shape)

```
order = {
    "order_id": "ORD-9001",
    "customer": {
        "name": "Ravi Patel",
```

```
        "email": "ravi@example.com",
    },
    "items": [
        {"sku": "P001", "name": "Notebook", "qty": 2, "price": 120},
        {"sku": "P003", "name": "Pen", "qty": 5, "price": 15},
    ],
    "status": "confirmed",
    "total": 315,
}

# Loop line items
for item in order["items"]:
    line_total = item["qty"] * item["price"]
    print(f"{item['name']}: ₹{line_total}")
```

Loop order items and print each line's total (qty × price).

```
order = {
    "customer": {"name": "Ravi", "email": "ravi@example.com"},
    "items": [
        {"sku": "P001", "qty": 2, "price": 120},
        {"sku": "P002", "qty": 1, "price": 499},
    ],
}
for item in order["items"]:
    print(f"₹{item['qty'] * item['price']}")
```

Print the customer's email from a nested order dict.

```
order = {
    "customer": {"name": "Ravi", "email": "ravi@example.com"},
    "items": [],
}
print(order["customer"]["email"])
```

Step 8 — Check membership and copy

```
user = {"username": "aisha", "role": "staff", "active": True}

print("username" in user)             # True — checks keys
print("admin" in user)                # False

# Shallow copy
user_backup = user.copy()
user_backup["role"] = "admin"
print(user["role"])                   # staff — original unchanged
```

Note:

- Wrong: Using `in` as if it checks values (`"aisha" in d` checks **keys** only).
- Prefer: `if "aisha" in d.values():` when you mean values.

Check if the email key exists in a contact dict and print Found or Missing.

```
contact = {"name": "Ravi", "phone": "9876543210"}
print("Found" if "email" in contact else "Missing")
```

Expected Output:

```
Missing
```

Common Error: `"Ravi" in contact` checks **keys**, not values — that expression is `False` even though `"Ravi"` is stored as a value.

Copy a user dict, change role on the copy, and show the original role unchanged.

```
user = {"username": "aisha", "role": "staff"}
backup = user.copy()
backup["role"] = "admin"
print(user["role"])    # staff
print(backup["role"])  # admin
```

Step 9 — Python dict ↔ JSON (preview for APIs)

```
import json

# Python dict → JSON string (for API response)
product = {"sku": "P001", "name": "Mouse", "price": 799}
json_text = json.dumps(product)
print(json_text)                    # {"sku": "P001", "name": "Mouse", "price": 799}

# JSON string → Python dict (from API request body)
parsed = json.loads(json_text)
print(parsed["name"])               # Mouse
```

Note:
- Wrong: Assuming JSON allows a tuple — JSON has no tuple type.
- Prefer: Convert tuples to lists before `json.dumps`.

Convert a product dict to a JSON string with `json.dumps` and print it.

```
import json

product = {"sku": "P001", "name": "Mouse", "price": 799}
print(json.dumps(product))
```

Parse JSON back to a dict with `json.loads` and print the product name.

```
import json

text = json.dumps({"sku": "P001", "name": "Mouse", "price": 799})
product = json.loads(text)
print(product["name"])
```

Practice Exercises

Basic

1. Create a dict for one student: `name`, `roll`, `marks`. Print each field using `.items()`.

Model answer:

```
student = {"name": "Dev", "roll": 21, "marks": 88}
for key, value in student.items():
    print(f"{key}: {value}")
```

2. Add key `"grade"` based on marks (A if ≥90, B if ≥75, else C).

Model answer:

```
student = {"name": "Dev", "roll": 21, "marks": 88}
marks = student["marks"]
if marks >= 90:
    student["grade"] = "A"
elif marks >= 75:
    student["grade"] = "B"
else:
    student["grade"] = "C"
print(student)
```

3. Use `.get()` to read `"phone"` with default `"Not provided"`.

Model answer:

```
student = {"name": "Dev", "roll": 21, "marks": 88}
phone = student.get("phone", "Not provided")
print(phone)  # Not provided
```

Intermediate

4. Build a product catalog: `{sku: {"name": ..., "price": ...}}` for 3 products. Lookup by SKU from user input.

Model answer:

```
catalog = {
    "P101": {"name": "Notebook", "price": 80.0},
    "P102": {"name": "Pen Set", "price": 120.0},
    "P103": {"name": "USB Drive", "price": 499.0},
}
sku = input("Enter SKU: ").strip().upper()
product = catalog.get(sku)
if product:
    print(f"{product['name']} — ₹{product['price']}")
else:
    print("SKU not found")
```

5. Count how many expenses per category: `[{"cat":"Food","amt":450}, ...]`.

Model answer:

```
expenses = [
    {"cat": "Food", "amt": 450},
    {"cat": "Travel", "amt": 120},
    {"cat": "Food", "amt": 200},
    {"cat": "Utilities", "amt": 900},
    {"cat": "Travel", "amt": 80},
]
counts = {}
for row in expenses:
    cat = row["cat"]
    counts[cat] = counts.get(cat, 0) + 1
print(counts)  # {'Food': 2, 'Travel': 2, 'Utilities': 1}
```

6. Update nested dict: change customer city inside an order dict.

Model answer:

```
order = {
    "order_id": "ORD-901",
    "customer": {"name": "Meera", "city": "Pune"},
    "total": 2499.0,
}
order["customer"]["city"] = "Bengaluru"
print(order["customer"])
```

Interview-style

7. Merge two dicts; if same key, **sum** numeric values (assume int/float).

Model answer:

```
def merge_sum(a, b):
    result = dict(a)
    for key, value in b.items():
        if key in result:
            result[key] = result[key] + value
        else:
            result[key] = value
    return result

left = {"Food": 450, "Travel": 120}
right = {"Food": 200, "Utilities": 900}
print(merge_sum(left, right))
# {'Food': 650, 'Travel': 120, 'Utilities': 900}
```

8. Invert dict `{101: "Ananya", 102: "Dev"}` → `{"Ananya": 101, "Dev": 102}` (assume unique values).

Model answer:

```
original = {101: "Ananya", 102: "Dev"}
inverted = {value: key for key, value in original.items()}
print(inverted)
```

9. Flatten one level: `{"a": 1, "b": {"c": 2, "d": 3}}` → keys `a`, `b.c`, `b.d` (optional stretch).

Model answer:

```
def flatten_one_level(data):
    flat = {}
    for key, value in data.items():
        if isinstance(value, dict):
            for inner_key, inner_val in value.items():
                flat[f"{key}.{inner_key}"] = inner_val
        else:
            flat[key] = value
    return flat

print(flatten_one_level({"a": 1, "b": {"c": 2, "d": 3}}))
# {'a': 1, 'b.c': 2, 'b.d': 3}
```

Mini-Project (Assignment): Contact Book CLI

Title: Phone-keyed contact book with CRUD
Lab kit (build here): 15_Lab_Assets/11_contact_book/

What to build

```
contacts = {}   # key: phone (str), value: dict with name, email

def add_contact():
    phone = input("Phone: ").strip()
    if phone in contacts:
        print("Contact exists. Use update.")
        return
    name = input("Name: ").strip()
    email = input("Email: ").strip()
    contacts[phone] = {"name": name, "email": email}
    print("Added.")

def search_contact():
    phone = input("Phone to search: ").strip()
    info = contacts.get(phone)
    if info:
        print(f"{info['name']} | {info['email']}")
    else:
        print("Not found.")

def list_contacts():
    for phone, info in contacts.items():
        print(f"{phone}: {info['name']} ({info['email']})")

# Menu: 1 Add  2 Search  3 List  4 Update  5 Delete  6 Exit
```

Stretch

- Search by name (scan values) when phone is unknown.
- Export contacts as a JSON string with `json.dumps`.

Done when

- [ ] Lab kit contact-book ticket add/search/list works
- [ ] Update and delete by phone work without KeyError
- [ ] Menu exits cleanly on option 6

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why are dict lookups often preferred over scanning a list for a matching ID?
2. How is a Python dict similar to a MySQL row + column names?
3. What happens if two API clients send the same JSON key twice?
4. When would nested dict become hard to maintain? (Preview: classes and methods.)
5. Can dict values be lists? Give a **Swiggy** order example.
6. **Dict vs list for lookup by ID?**
   A: Dict O(1) average vs list O(n) scan.
7. **Are dict keys ordered?**
   A: Insertion order preserved since Python 3.7; still treat order as a bonus, not the main reason to use a dict.
8. **Copy dict?**
   A: `.copy()` shallow; `copy.deepcopy()` for nested.
9. **dict comprehension example?**
   A: `{k: v*2 for k, v in d.items()}`.
10. **Can we sort a dict?**
    A: Sort keys: `sorted(d)`; items: `sorted(d.items(), key=...)`.